

1 DataLeakDetector 项目介绍与架构说明

生成日期: 2026-06-30

分支: h

参考文档: docs/introduce.md、docs/record.md

最近架构提交: 2a31428、90e338c、b3fb78d

1.1 项目定位

DataLeakDetector 是一个面向桌面办公场景的数据泄露检测系统。它综合使用系统日志、窗口标题、录屏关键帧、OCR/VLM 视觉分析和 Datalog 符号推理, 还原敏感数据从“被打开”到“被派生、复制、上传、截图、屏幕共享或物理转移”的完整证据链。

项目当前的核心目标不是简单判断“用户是否打开过敏感文件”, 而是回答三个可审计问题:

- 敏感对象是什么: 原始文件、派生文件、内容片段或截图/录屏内容。
- 数据发生了什么: 查看、复制、另存、压缩、重命名、上传、发送、同步、屏幕共享等。
- 是否离开可信环境: 邮件、AI 服务、聊天工具、网盘、代码仓库、会议共享、U 盘、HTTP POST 等。

全量样例 211 NAS benchmark 已跑完全量	VLM 复核 146 live VLM review cases	最终 F1 63.6% VLM 后最终判定; precision 93.3%
---	---	---

1.2 当前文件架构

当前仓库保留了早期原型目录和新架构目录。为了不打断已有脚本, 本轮没有大规模重命名旧目录, 而是采用“兼容目录 + 新职责说明”的方式整理架构边界。

目录	当前定位	说明
ScreenMonitor/	日志与录屏采集	Windows/macOS 监控端, 产生文件、窗口、剪贴板、截图、录屏等原始证据。
2-EventCorrelator/	新日志挖掘层	对应 docs/introduce.md 中的 EventCorrelator 职责: 前台应用识别、敏感窗口构建、多跳 lineage、关联候选输出。
1-FrameAnalyzer/	视频关键帧与 VLM 分析	历史目录名仍为模块 1; 在目标架构中承担 FrameAnalyzer 职责, 负责 OCR、关键帧筛选和 VLM 结构化输出。
2-FileTracker/	旧派生追踪原型	保留旧行为分析图和 worklist 逻辑; 新派生链核心能力逐步迁移到 2-EventCorrelator/event_correlator/lineage.py。
3-RiskHunter/	风险编排与 log-first 检测	现阶段主入口之一, 包含 LogFirstDetector、VLM fallback gate 和可疑窗口构建。
4-ThreatDetector/	符号推理	Datalog 推理层, 对 OpenFile、TransferFile、CrossProcessTransfer、LeakFile 等事实进行泄露路径推断。
tools/	评估与数据工具	NAS 数据下载、benchmark、表格生成、离线回归等工具。
reports/	报告与实验结果	Typst 报告、NAS 中间结果、全量 VLM 表格和分支总结。

文件架构处理原则

当前最重要的是不降低检测准确率和不破坏历史脚本。因此顶层目录暂不强行改成 `modules/01-*` 形式，而是在报告与 README 中明确“历史目录名”和“目标职责”的映射。后续若要彻底迁移，应先增加统一包入口，再分阶段更新 `sys.path`、运行脚本和文档链接。

1.3 目标架构

`docs/introduce.md` 中的目标架构可以收敛为三层感知与一层推理：

DataLeakDetector 总体架构

日志先行缩小搜索空间，视觉模型确认完成态，符号推理输出可审计的泄露路径

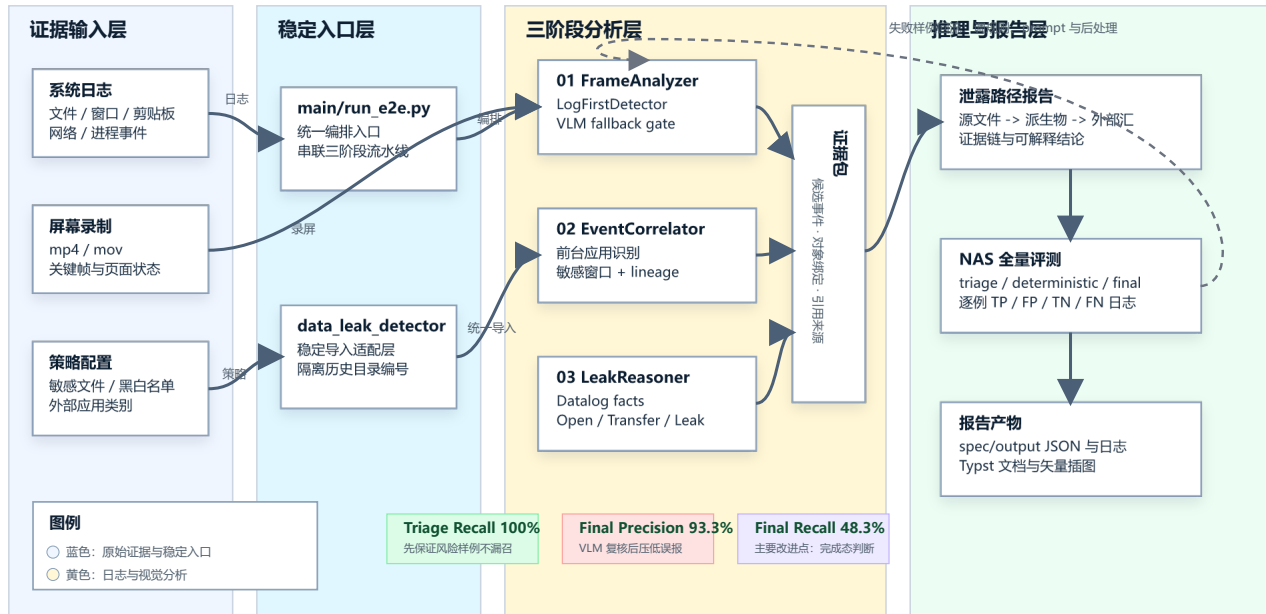


图 1 DataLeakDetector 总体架构：采集层提供日志和录屏，EventCorrelator 生成敏感窗口和派生链，FrameAnalyzer 在窗口内做 OCR/VLM 取证，LeakReasoner 以 Datalog 输出可审计泄露路径。

完整数据流如下：

ScreenMonitor 日志/录屏

- > EventCorrelator：日志入库、FrontendApp、敏感窗口、lineage
- > FrameAnalyzer：OCR 缓存、关键帧选择、VLM 行为分析
- > EventCorrelator：透传+相关性标注、派生追踪、多轮补跑窗口
- > LeakReasoner：Datalog fact、符号推理、证据链报告

在现有代码中，这条目标链路以兼容方式落地：

- 3-RiskHunter/log_first_detector.py 提供确定性日志检测。
- 3-RiskHunter/suspicious_window_builder.py 提供 VLM fallback gate。
- 2-EventCorrelator/event_correlator/frontend.py 解析 window_info.window_title，把浏览器窗口进一步标注为 email、ai_service、cloud_storage、meeting 等。
- 2-EventCorrelator/event_correlator/windows.py 构建 analysis_windows。
- 2-EventCorrelator/event_correlator/lineage.py 支持沿已知 artifact 的多跳派生追踪。
- 4-ThreatDetector/datalog/ 将事件落到 Datalog 推理。

1.4 日志挖掘层：EventCorrelator

EventCorrelator 的任务是系统先从确定性日志中提取最大信息量，再把确实需要视觉语义的片段交给 FrameAnalyzer。

轻量证据图模型

把日志、窗口、文件派生、视觉证据和 Datalog facts 统一成可追溯的审计链

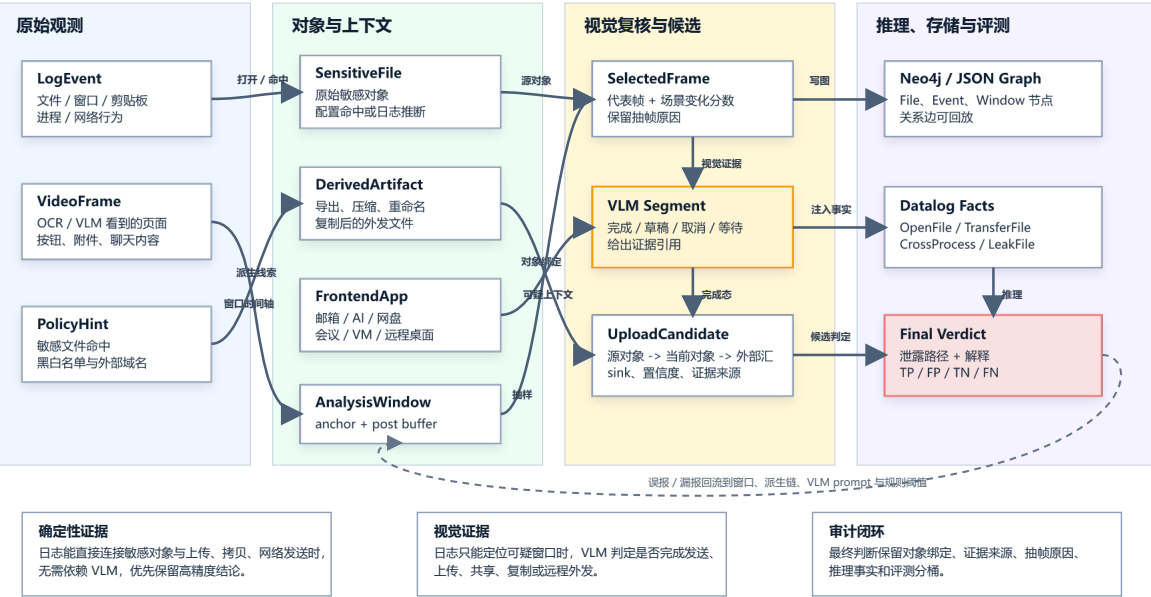


图 2 轻量证据图谱: Process、Event、File、Window、FrontendApp、ContentArtifact 和 External Sink 之间的关系, 为 Neo4j 写回和 Datalog 推理提供统一结构。

能力	输入	输出
前台应用识别	window_info.window_title、URL、浏览器进程名。	FrontendApp 类别, 例如 email、ai_service、cloud_storage、code_repo、meeting。
敏感窗口构建	敏感文件路径、文件名、窗口标题、关键词、系统日志时间戳。	analysis_windows, 包含窗口起止时间、anchor 文件、外部前台应用、候选事件和 post buffer。
派生链构建	文件 open/create/rename/copy/compress/convert/upload 事件, 以及 VLM 行为段。	direct_file_mappings、full_file_mapping_chains、artifact_instances。
候选外发生成	日志事件、VLM segment、lineage、sink 类型。	UploadCandidate, 包含原始敏感文件、当前外发文件、sink、证据引用和 object binding。

1.4.1 前台应用识别

浏览器进程名本身信息很弱, msedge.exe 不能说明用户是在邮箱、AI、网盘还是普通搜索页面。当前实现会解析窗口标题和 URL:

```
{
  "window_info": {
    "window_title": "mail.163.com 和另外 1 个页面 - 个人 - Microsoft Edge Beta"
  }
}
```

会被归类为:

```
{
  "category": "email",
  "display_name": "email:mail.163.com 和另外 1 个页面",
  "is_external": true
}
```

这一步的收益是: 日志层可以知道“外部前台应用”是否出现, 从而构建更精准的视频分析窗口, 而不是把所有浏览器活动都当成同一种风险。

1.4.2 敏感窗口构建

敏感窗口遵循 docs/introduce.md 的口径：

- 按 exact_path、filename、window_title、derived_under_sensitive_stem_dir、keyword 找敏感 anchor。
- 对同一敏感实体聚合首次事件到末次事件。
- 查询时间段内非白名单应用共现事件。
- 查询后续外部 FrontendApp 活动。
- 增加 post_buffer_seconds=10，避免刚好漏掉发送完成态。

输出示例：

```
{
  "window_id": "sensitive_window_1",
  "sensitive_file": "C:/work/customer.xlsx",
  "start": "2026-01-01 10:00:00",
  "end": "2026-01-01 10:00:40",
  "match_types": ["exact_path", "filename"],
  "frontend_categories": ["email"],
  "post_buffer_seconds": 10
}
```

1.4.3 多跳派生追踪

旧链路容易只追一轮派生，例如 customer.xlsx -> customer.pdf 能追到，但 customer.pdf -> customer.zip -> U 盘 容易断链。当前 LineageBuilder 已支持沿 “已知 artifact 集合” 继续推断：

```
customer.xlsx
-> customer.pdf
-> customer.zip
-> E:/customer.zip
```

同时导出 artifact_instances：

```
{
  "artifact_id": "c:/work/customer.zip@2026-01-01 10:00:20",
  "path": "C:/work/customer.zip",
  "parent_path": "C:/work/customer.pdf",
  "root_path": "C:/work/customer.xlsx",
  "event_type": "compressed"
}
```

artifact_id = normalized_path + nearest_evidence_time，用于区分同路径同名但不是同一个文件的派生实体。

1.5 FrameAnalyzer：窗口内视频取证

FrameAnalyzer 的职责是在 EventCorrelator 给出的 analysis_windows 内做低成本视觉分析。它不是对整段视频无差别调用 VLM，而是先进行采样、OCR 和本地候选筛选。

阶段	筛选依据	目的
窗口抽帧	在 analysis_windows 内按 1fps 或锚点优先采样。	避免处理整段录屏。
OCR 缓存	保存每帧 OCR 文本、图片路径和元数据。	复用 OCR 结果，减少重复计算。
候选筛选	文件名命中、敏感关键词、上传/发送/成功/取消状态词、帧差、OCR 变化、日志事件附近帧。	让 VLM 看到最可能有证据的帧。
Window Bundle	按 “日志窗口 -> 应用阶段 -> 行为阶段” 组织上下文。	让模型一次输出多个行为事件，减少调用次数。
VLM 输出	活跃应用、UI 场景、动作信号、风险阶段、可见文件、派生对象、证据原因。	生成结构化行为事件，供 EventCorrelator 回写和 LeakReasoner 推理。

VLM 输出期望区分三类：

- 正常操作：查看、编辑、内部白名单处理。
- 潜在隐藏行为：复制、截图、OCR 提取、导出、压缩、重命名、内容转换。
- 直接外发：邮件发送、网盘上传、聊天发送、表单提交、屏幕共享暴露。

对于潜在隐藏行为，需要额外说明 artifact 转换类型：

```
{
  "derivative_type": "file",
  "source_artifact_type": "file",
  "target_artifact_type": "file",
  "trackable": true,
  "derived_files": ["customer.zip"]
}
```

1.6 LeakReasoner: 符号推理与证据链

LeakReasoner 将上游结果转为 Datalog fact，并推理完整泄露路径。相比单条告警，它更强调“为什么判定泄露”。

```
OpenFile(user_app, customer.xlsx)
TransferFile(user_app, customer.xlsx -> customer.zip)
CrossProcessTransfer(user_app -> browser, customer.zip)
LeakFile(browser, customer.zip, network)
```

最终输出应包含：

- 原始敏感文件。
- 派生文件或内容 artifact。
- 外发应用或外部设备。
- 时间范围与证据 segment。
- 置信度与推理路径。

1.7 VLM 触发逻辑

当前系统不是所有 case 都用 VLM。判断流程如下：

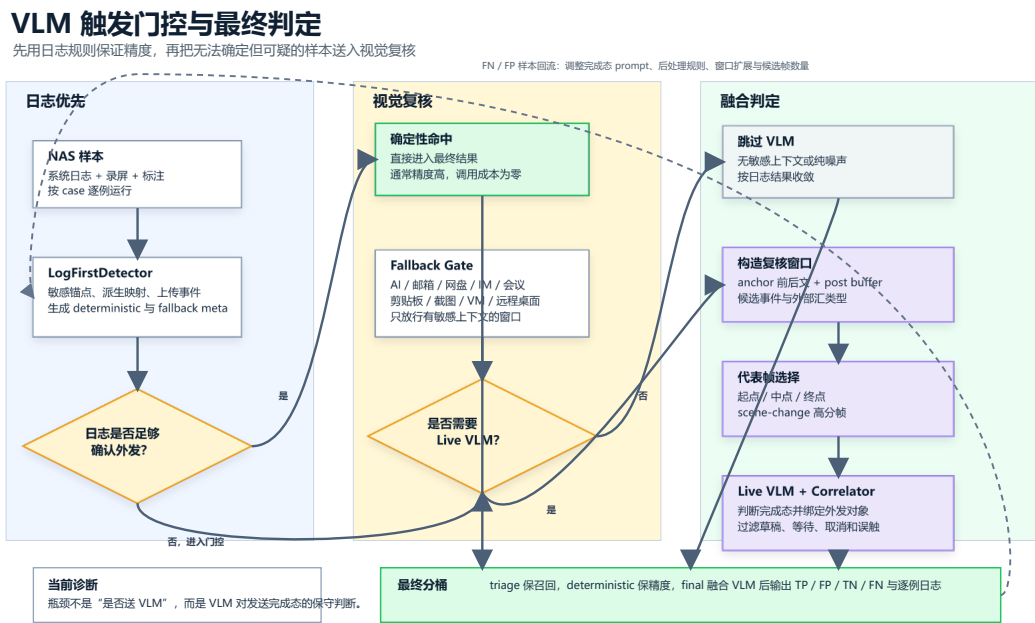


图 3 VLM 触发门控：日志证据完整时直接输出；日志无法确认但存在敏感上下文和模糊外发信号时才抽帧调用 VLM；无敏感上下文或噪声样例直接跳过。

触发 VLM 的典型信号：

- 敏感文件或敏感文本已经在日志中出现。
- 附近存在 AI、邮箱、聊天、网盘、会议、剪贴板、截图、录屏、上传、发送、附件等上下文。

- 或出现派生行为：created、modified、renamed、copied、compressed、converted、clipboard_text、screenshot_capture。
- 事件处于敏感 anchor 的时间窗口附近。
- 系统噪声路径和白名单应用被过滤。

如果日志能确定外发，则 VLM 被跳过；如果既无敏感上下文，也无模糊外发动作，也跳过 VLM。

1.8 全量 VLM 评估结果

本轮使用 .env 中配置的 VLM 对 NAS 全量数据运行：

```
python tools/benchmark_nas_samples.py --use-vlm --json-output output/nas_full_vlm_report.json
python tools/report_benchmark_table.py output/nas_full_vlm_report.json
```

生成文件：

- output/nas_full_vlm_report.json
- output/nas_full_vlm_table.md
- output/nas_full_vlm_table.json
- output/nas_full_vlm_stdout.log
- output/nas_full_vlm_stderr.log

TABLE 4: Single-step exposure detection across held-out applications (RQ1).

Category	#Case	#Apps	#Held	Prec(%)	Recall (%)	F1 (%)
Email	21	4	2	100.0	75.0	85.7
AI Chat	19	4	2	83.3	62.5	71.4
IM	19	4	2	100.0	77.8	87.5
Cloud Drive	20	4	2	100.0	55.6	71.4
Meeting	16	3	2	85.7	60.0	70.6
Collaboration	19	5	2	80.0	50.0	61.5
Technical Forum	18	4	2	66.7	33.3	44.4
Code Hosting	18	4	2	100.0	66.7	80.0
Overall	150	32	16	91.7	62.0	73.9

图 4 参考论文表格样式：按类别展示 Case、Apps、Held-out、Precision、Recall、F1。

Category	#Case	#Apps	#Held	Prec(%)	Recall(%)	F1(%)
AI Chat	19	10	5	100.0	64.3	78.3
Bluetooth	3	2	0	100.0	66.7	80.0
Cloud Drive	18	11	5	88.9	61.5	72.7
Code Hosting	17	8	4	100.0	61.5	76.2
Collaboration	18	9	4	100.0	35.7	52.6
Content	9	4	0	100.0	25.0	40.0
Content Transform	7	2	0	100.0	71.4	83.3
E2E	24	1	0	100.0	50.0	66.7
Email	19	8	5	71.4	35.7	47.6
File Structure	12	4	0	100.0	45.5	62.5
IM	15	7	4	88.9	72.7	80.0
Meeting	15	6	3	66.7	33.3	44.4
Screen	8	2	0	100.0	37.5	54.5

Steganography	3	2	0	100.0	0.0	0.0
Technical Forum	14	10	4	100.0	40.0	57.1
Transfer	7	4	0	100.0	66.7	80.0
Virtual Machine	3	3	0	100.0	0.0	0.0
Overall	211	88	33	93.3	48.3	63.6

解释：

- #Held 按本地数据集中 0-normal-* hold-out/正常应用覆盖数统计。
- triage 指 “日志确定 + 需要 VLM 的候选”，当前 recall 为 100%。
- final 指 VLM 复核后的最终判定，precision 提升到 93.3%，但 recall 降到 48.3%。
- 当前主要瓶颈不在 “是否送 VLM”，而在 VLM 完成态判定过保守。

1.9 当前问题与改进计划

问题	表现	计划
目录编号过渡	当前存在 1-FrameAnalyzer、2-FileTracker、2-EventCorrelator 等历史编号，和目标三模块架构并非完全一致。	先保留兼容；后续建立统一 modules/ 包入口，再分阶段迁移脚本和 README。
VLM 召回不足	全量 VLM final recall 为 48.3%，大量正例被完成态规则或模型判断压成 false。	对 FN 样例做类别分析，调整邮件/AI/网盘/会议的完成态 prompt 和后处理策略。
Neo4j 尚未完全落地	当前轻量图结构主要在内存 bundle 和 JSON 中，尚未统一写入 Neo4j。	增加 graph writer，落地 File、Event、Window、FrontendApp、ContentArtifact 节点和证据边。
多轮追踪还需补强	已支持多跳 lineage，但 artifact follow-up 自动补跑窗口仍未完整闭环。	实现 closed/terminal/needs_followup 状态机，限制 max rounds 和 max windows。
评估口径需固定	当前表格按样本名推断类别和 App，仍需与数据集元数据对齐。	后续补充 dataset manifest，显式记录 category、app、held_out、positive/negative。

1.10 总结

当前项目已经从“主要依赖视频/VLM 判断”推进到“日志优先、视觉兜底、符号推理收束”的架构。EventCorrelator 负责把系统日志整理成敏感窗口、前台应用、派生链和上传候选；FrameAnalyzer 在窗口内用 OCR/VLM 做语义确认；LeakReasoner 通过 Datalog 给出可审计的泄露路径。

本轮文件架构处理采用低风险策略：清理生成缓存，新增清晰的 Typst 架构报告，并在文档中明确历史目录到目标架构的映射。这样既不破坏现有 benchmark 和运行脚本，也为后续模块迁移、Neo4j 写回和多轮 artifact 追踪留下明确路径。